

T2.1.D · Reservas de créditos, descuento real, audit log extendido y pruning seguro

Owner: Celestino · 2026-05-06 · branch *pr/t2.1.d-credit-reservations-audit-pruning* · cambios mínimos · sin deploy · sin cambios externos · sin borrado de datos productivos.

Resumen ejecutivo: el ejecutor de acciones del Action Center ahora reserva créditos al iniciar la ejecución, los descuenta del saldo del usuario, libera (reembolsa) si la ejecución falla y confirma si tiene éxito. Todo el flujo es idempotente: re-llamar *reservar()* para el mismo *accion_id* NO duplica el cobro. Se sumaron 6 eventos al audit log (whitelist) y se añadió un módulo de *pruning* con **dry-run por defecto**, *max_delete* tope y validación de antigüedad mínima (30 días acciones, 90 días audit). Endpoints admin: *GET /admin/automation/prune/preview* y *POST /admin/automation/prune* (requiere *confirm=BORRAR*).

1. Causa y diseño

1.1 Problema que resuelve

T2.1.A introdujo el motor de ejecución del Action Center. T2.1.B sumó dashboard y endpoints. T2.1.C conectó ejecutores reales con LLM y fallback determinístico. Pero hasta T2.1.C la ejecución NO descontaba créditos: el costo se estimaba (*agent.automation.costos.estimar_costo_accion*) pero nunca se cobraba. Además el audit log no registraba ni reservas ni bloqueos por saldo, y no había forma segura de limpiar tablas viejas. T2.1.D cierra estas tres brechas.

1.2 Decisión de diseño · cobrar al crear / reembolsar si falla

Se evaluaron dos alternativas: **(a) lockear saldo** (crear una fila “hold” sin descontar y consolidar al final) vs **(b) cobrar y reembolsar**. Se eligió (b) porque *agent.billing.cobrar()* ya implementa el descuento atómico con CAS (compare-and-set) anti-race y *acreditar()* deja audit-trail en *transacciones_credito*. Lockear duplicaría lógica de concurrencia y rompería el invariante “saldo entero consistente con suma de transacciones”. El reembolso queda registrado como *credit_reservation_release* en la misma tabla, manteniendo un único punto de verdad.

1.3 Idempotencia · estado-máquina + UNIQUE(accion_id)

Cada ejecución tiene exactamente una fila en *automation_reservas_credito* con UNIQUE(accion_id). El estado es una máquina finita:

Estado	Trigger	Efecto en saldo	Terminal
pending	reservar() exitoso	descuenta créditos	No
confirmed	confirmar() tras ejecución OK	—	Sí
released	liberar() tras error / bloqueo	acredita créditos	Sí
failed	reservar() con saldo insuficiente	—	Sí

Re-llamar *reservar(accion_id=42)* con una fila *pending* existente retorna la fila tal cual, sin invocar *billing.cobrar()* de nuevo. Igual para *confirmar()* sobre una fila ya *confirmed*: noop. Por eso el reintento del worker (que es esperable) NO genera doble cobro.

2. Archivos modificados y creados

Archivo	Tipo	Resumen
---------	------	---------

agent/automation/credits.py	NUEVO ~280 ln	API <i>reservar / confirmar / liberar / obtener_reserva</i> . Excepción <i>Credito</i>
agent/automation/models.py	MODIFICADO	Modelo <i>ReservaCreditoAutomation</i> + 4 entradas en <i>MIGRACIONES</i>
agent/automation/audit.py	MODIFICADO	<i>EVENTOS_VALIDOS</i> extendido con 6 eventos: <i>credits_reserved</i> · <i>credits</i>
agent/automation/execution.py	MODIFICADO	<i>ejecutar_accion()</i> ahora: <i>reserva</i> → <i>marca_running</i> → <i>ejecuta</i> → <i>confirmar</i>
agent/automation/pruning.py	NUEVO ~250 ln	<i>pruning_acciones</i> · <i>pruning_reservas</i> · <i>pruning_audit_log</i> . <i>dry_run</i> por de
agent/main.py	MODIFICADO	Endpoints admin: GET /admin/automation/prune/preview (dry-run coun
tests/test_automation_creditos_reservas.py	NUEVO 19 tests	Cobertura de reserva directa, idempotencia, confirmar/liberar, ejecución integra
tests/test_automation_pruning.py	NUEVO 13 tests	Cobertura de dry-run, ejecución, anti-borrado-corto, no borra in-flight ni reciente
tests/test_automation_execution.py	MODIFICADO	Fixture <i>monkeypatch</i> de <i>estimar_costo_accion</i> → 0 antes y despu
tests/test_automation_executors_real.py	MODIFICADO	Mismo monkeypatch que arriba (T2.1.C legacy).
tests/test_automation_endpoints.py	MODIFICADO	Mismo monkeypatch que arriba (T2.1.B legacy).

3. Migración de datos

Se sumaron 4 entradas a *MIGRACIONES_AUTOMATION* en *agent/automation/models.py*. El motor de migraciones de Dona (lista *_MIGRACIONES* en runtime, no Alembic) evalúa cada sentencia con *IF NOT EXISTS*, lo que las hace idempotentes en reentrega y compatibles con SQLite (tests) y Postgres (prod).

```
-- automation_reservas_credito
CREATE TABLE IF NOT EXISTS automation_reservas_credito (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  accion_id INTEGER NOT NULL UNIQUE,
  telefono VARCHAR(50) NOT NULL,
  credits INTEGER NOT NULL,
  estado VARCHAR(20) NOT NULL DEFAULT 'pending',
  razon TEXT NOT NULL DEFAULT '',
  transaccion_credito_id INTEGER,
  creado DATETIME NOT NULL,
  actualizado DATETIME NOT NULL
);
CREATE UNIQUE INDEX IF NOT EXISTS
  ix_automation_reservas_credito_accion_id
  ON automation_reservas_credito(accion_id);
CREATE INDEX IF NOT EXISTS
  ix_automation_reservas_credito_telefono
  ON automation_reservas_credito(telefono);
CREATE INDEX IF NOT EXISTS
  ix_automation_reservas_credito_estado
  ON automation_reservas_credito(estado);
```

4. Anti doble cobro · evidencia

Tres capas de defensa contra el doble cobro:

Capa	Mecanismo	Test que lo valida
DB	UNIQUE constraint sobre <i>accion_id</i>	<code>test_no_duplica_cobro_si_se_llama_dos_veces</code>
App	<code>reservar()</code> chequea fila existente antes de cobrar	<code>test_re_ejecutar_no_doble_cobra</code>
Audit	evento <i>credits_reserved</i> sólo se emite cuando el cobro <i>no se reservó antes</i>	<code>test_reservar_emite_eventos_audit</code>

Adicional: si la ejecución falla (excepción, CRITICAL bloqueado o HIGH sin ejecutor), *liberar()* reembolsa con *billing.acreditar()* y marca la fila como *released*. El reembolso emite *credits_released* en audit, dejando trazabilidad completa: cobro → reembolso, ambos vinculados al mismo *accion_id*.

5. Audit log extendido

Se añadieron 6 eventos a la whitelist *EVENTOS_VALIDOS*:

Evento	Cuándo se emite	Carga útil resumen
<code>credits_reserved</code>	<code>reservar()</code> exitoso	tipo · credits · accion_id
<code>credits_confirmed</code>	<code>confirmar()</code> tras ejecución OK	tipo · credits · accion_id
<code>credits_released</code>	<code>liberar()</code> tras error o bloqueo	tipo · credits · accion_id · razon
<code>credits_reservation_failed</code>	saldo insuficiente en <code>reservar()</code>	tipo · requerido · saldo · accion_id
<code>action_blocked_insufficient_credits</code>	<code>ejecutar_accion()</code> abortado por <code>reservar()</code> fallido	tipo · accion_id
<code>pruning_executed</code>	pruning ejecutado (no dry-run)	tabla · borradas · dias · ts_inicio · ts_fin

Todos los eventos pasan por *audit.registrar()* que trunca *telefono* a ****últimos4* y serializa *payload_summary* con saneado (sin secrets ni PII de contacto). Test *test_evento_no_filtra_pii* valida que no aparezca el teléfono completo en los payloads.

6. Verificaciones · comandos y resultados

6.1 Tests específicos T2.1.D

```
pytest tests/test_automation_creditos_reservas.py -v
→ 19 passed in 14.89s

pytest tests/test_automation_pruning.py -v
→ 13 passed in 9.53s
```

6.2 Tests automation completos (sin regresión)

```
pytest tests/test_automation_*.py
→ 197 passed in 107.51s
```

6.3 Suite completa del repo

```
pytest
→ 859 passed in 221.80s
```

0 failed · 0 regresión. Los warnings restantes son *PytestUnhandledThreadExceptionWarning* del cleanup de aiosqlite (event-loop closed), ya presentes en main y ortogonales a T2.1.D.

7. Riesgos residuales y mitigaciones

Riesgo	Severidad	Mitigación
Crash entre <code><i>billing.cobrar()</code> </i> y INSERT en reserva_credito (descuento sin fila)	Bajo	Desierento en fila, el prointervalo deja saldo descontado pero sin reserva visible. El
Worker reintentta ejecutar_accion() tras éxito ya con Bajeado	Bajo	<code><i>confirmar()</code> </i> sobre fila <code><i>confirmed</i></code> es noop · no re-emite audit ni re-t
Pruning borra más de lo esperado	Medio	dry-run por defecto, <code><i>ejecutar_borrado=True</i></code> requerido explícitamente, <code><i></code>
Cambios de schema en prod (PG) al primer redeploy	Bajo	<code><i>CREATE TABLE IF NOT EXISTS</i></code> idempotente. La tabla nueva no afecta
Latencia adicional por <code><i>cobrar()</code> </i> en cada ejecución	Bajo	Un INSERT + UPDATE atómicos (~5-10ms PG). Acciones del Action Center se

8. Instrucciones de PR

Branch: `pr/t2.1.d-credit-reservations-audit-pruning`

Título: `feat(automation): add credit reservations, real deduction, audit logs and pruning`

Cuerpo sugerido:

```
## Resumen
T2.1.D cierra el flujo de billing real para el Action Center:
· reserva atómica con cobro inmediato y refund automático
· 6 nuevos eventos en audit log (whitelist)
· módulo pruning seguro (dry-run default, dias min, max-cap)
· 2 endpoints admin (preview + execute con confirm=BORRAR)

## Cambios
· NUEVO agent/automation/credits.py (~280 ln)
· NUEVO agent/automation/pruning.py (~250 ln)
· MOD agent/automation/models.py (+modelo +4 migraciones)
· MOD agent/automation/audit.py (+6 eventos)
· MOD agent/automation/execution.py (reserva/confirma/libera)
· MOD agent/main.py (+2 endpoints admin)
· NUEVO tests/test_automation_creditos_reservas.py (19)
· NUEVO tests/test_automation_pruning.py (13)
· MOD 3 fixtures de tests legacy con monkeypatch costo=0
```

```
## Verificaciones
· pytest 859 passed
· pytest tests/test_automation_* 197 passed
· sin deploy · sin cambios externos · sin borrado prod

## Guardrails preservados
· CRITICAL sigue bloqueado y libera reserva
· MEDIUM/HIGH requieren aprobación
· anti-IDOR intacto en endpoints admin (Bearer token)
· audit log con telefono truncado y payload sanitizado
```

9. Próximo paso recomendado

Con T2.1.D cerrado, el motor de automation tiene billing real, audit completo y mantenimiento operativo. El siguiente bloque natural es **T2.1.E · scheduler periódico** (cron-like) para detectar oportunidades de acción de forma proactiva, o bien **T2.2 · primera acción HIGH real** (e.g. enviar_mensaje_whatsapp con conexión a Whapi en preview-then-commit). La decisión depende de cuánto tráfico orgánico estamos viendo en el Action Center vs cuánto necesitamos demostrar valor de envío real.